

Asignatura	Datos del alumno	Fecha
Generalización de Código de Automatización en Desarrollo de Software con IA	Apellidos: Vegas Salazar	29/06/2026
	Nombre: Juan Carlos	

Actividad 2. Generación de entornos de pruebas y casos de test asistidos por IA

URL PROD: <https://latamtradex-actividad2.azucarsintactica.com/?demo=1>

GITHUB: <https://github.com/juank704/latamtradex-generativecode/tree/feature/test-activity-2>

DOCUMENTACION IA: https://github.com/juank704/latamtradex-generativecode/blob/feature/test-activity-2/docs/registro_prompts_actividad_2.txt

Resumen del alcance de las pruebas

El objetivo de esta fase fue garantizar la fiabilidad de las funcionalidades más críticas de *Latamtradex* mediante pruebas unitarias y de integración. Para ello, se solicitó a la IA generar casos de prueba focalizados en cuatro áreas principales:

1. **Esquemas de validación (Zod):** Pruebas unitarias para verificar que la entrada de datos en registros y cotizaciones (métodos de pago) rechace datos inválidos.
2. **Modelo de Roles y Autenticación:** Verificación de que la sesión JWT distinga correctamente entre roles (ADMIN, PROVIDER, BUYER) y bloquee el acceso no autorizado.

Asignatura	Datos del alumno	Fecha
Generalización de Código de Automatización en Desarrollo de Software con IA	Apellidos: Vegas Salazar	29/06/2026
	Nombre: Juan Carlos	

3. **Máquina de Estados de la Orden de Compra:** Validación de la lógica de negocio, asegurando que el flujo (GENERATED → SCHEDULED → PREPARING, etc.) no permita transiciones inválidas o saltos de pasos.
4. **Componentes UI (Panel del Comprador):** Pruebas de integración usando *React Testing Library* para verificar que la línea de tiempo visualice correctamente el estado de la orden.
5. **Pruebas End-to-End (E2E) con Playwright:** Se implementó una suite de pruebas de interfaz de usuario y flujos completos simulando el comportamiento real de los usuarios en el navegador. Se evaluó el "Happy Path" completo: desde que un proveedor publica un producto, el administrador lo aprueba, el comprador solicita cotización, hasta la generación de la Orden de Compra y su seguimiento interactivo, ejecutando todo de forma *headless* dentro del contenedor Docker.

Asignatura	Datos del alumno	Fecha
Generalización de Código de Automatización en Desarrollo de Software con IA	Apellidos: Vegas Salazar	29/06/2026
	Nombre: Juan Carlos	

Capturas y logs de ejecución

```
PS C:\Users\jcvieg\Desktop\repo_local\master_universitario_2026\generaciondecodigo\actividad1\latamtradox-generativecode> docker run --rm latamtradox-builder npm run test:coverage
> latamtradox@0.1.0 test:coverage
> jest --coverage

jest-haste-map: Haste module naming collision: latamtradox
  The following files share their name; please adjust your hasteimpl:
    * crootdir>.next/standalone/package.json
    * crootdir>package.json

PASS tests _purchaseOrderStateMachine.test.ts
PASS tests _auth.roles.test.ts
PASS tests _validation.test.ts
PASS tests _buyerPanelTimeline.test.tsx

File | % Stats | % Branch | % Funcs | % Lines | Uncovered Line #s
-----|-----|-----|-----|-----|-----
All files | 7.88 | 43.52 | 21.05 | 7.88 |
app | 0 | 0 | 0 | 0 |
page.tsx | 0 | 0 | 0 | 0 | 1-114
app/api/auth/login | 0 | 0 | 0 | 0 |
route.ts | 0 | 0 | 0 | 0 | 1-31
app/api/auth/logout | 0 | 0 | 0 | 0 |
route.ts | 0 | 0 | 0 | 0 | 1-7
app/api/auth/register | 0 | 0 | 0 | 0 |
route.ts | 0 | 0 | 0 | 0 | 1-47
app/api/checkout | 0 | 0 | 0 | 0 |
route.ts | 0 | 0 | 0 | 0 | 1-76
app/api/documents | 0 | 0 | 0 | 0 |
route.ts | 0 | 0 | 0 | 0 | 1-71
app/api/documents/[id] | 0 | 0 | 0 | 0 |
route.ts | 0 | 0 | 0 | 0 | 1-65
app/api/products | 0 | 0 | 0 | 0 |
route.ts | 0 | 0 | 0 | 0 | 1-39
app/api/products/[id] | 0 | 0 | 0 | 0 |
route.ts | 0 | 0 | 0 | 0 | 1-60
app/api/purchase-orders/[id] | 0 | 0 | 0 | 0 |
route.ts | 0 | 0 | 0 | 0 | 1-92
app/api/quotes | 0 | 0 | 0 | 0 |
route.ts | 0 | 0 | 0 | 0 | 1-54
app/api/quotes/[id] | 0 | 0 | 0 | 0 |
route.ts | 0 | 0 | 0 | 0 | 1-74
app/asesorias | 0 | 0 | 0 | 0 |
CheckoutButton.tsx | 0 | 0 | 0 | 0 | 1-40
page.tsx | 0 | 0 | 0 | 0 | 1-68
app/catalogo | 0 | 0 | 0 | 0 |
```

```
PS C:\Users\jcvieg\Desktop\repo_local\master_universitario_2026\generaciondecodigo\actividad1\latamtradox-generativecode> docker run --rm latamtradox-builder npm run test:coverage
app/catalogo | 0 | 0 | 0 | 0 | 1-84
page.tsx | 0 | 0 | 0 | 0 |
app/checkout/success | 0 | 0 | 0 | 0 |
page.tsx | 0 | 0 | 0 | 0 | 1-72
app/dashboard/admin | 0 | 0 | 0 | 0 |
page.tsx | 0 | 0 | 0 | 0 | 1-91
app/dashboard/admin/cotizaciones | 0 | 0 | 0 | 0 |
QuoteAdminList.tsx | 0 | 0 | 0 | 0 | 1-104
page.tsx | 0 | 0 | 0 | 0 | 1-48
app/dashboard/admin/documentos | 0 | 0 | 0 | 0 |
DocumentModeration.tsx | 0 | 0 | 0 | 0 | 1-115
page.tsx | 0 | 0 | 0 | 0 | 1-41
app/dashboard/admin/productos | 0 | 0 | 0 | 0 |
ProductModeration.tsx | 0 | 0 | 0 | 0 | 1-131
page.tsx | 0 | 0 | 0 | 0 | 1-44
app/dashboard/admin/usuarios | 0 | 0 | 0 | 0 |
page.tsx | 0 | 0 | 0 | 0 | 1-56
app/dashboard/comprador | 27.16 | 93.75 | 50 | 27.16 |
PurchaseOrderTracker.tsx | 100 | 100 | 100 | 100 |
page.tsx | 0 | 0 | 0 | 0 | 1-177
app/dashboard/comprador/asesorias | 0 | 0 | 0 | 0 |
page.tsx | 0 | 0 | 0 | 0 | 1-72
app/dashboard/proveedor | 0 | 0 | 0 | 0 |
page.tsx | 0 | 0 | 0 | 0 | 1-85
app/dashboard/proveedor/documentos | 0 | 0 | 0 | 0 |
DocumentManager.tsx | 0 | 0 | 0 | 0 | 1-186
page.tsx | 0 | 0 | 0 | 0 | 1-49
app/dashboard/proveedor/ordenes | 0 | 0 | 0 | 0 |
PurchaseOrderManager.tsx | 0 | 0 | 0 | 0 | 1-181
page.tsx | 0 | 0 | 0 | 0 | 1-50
app/dashboard/proveedor/productos | 0 | 0 | 0 | 0 |
ProductManager.tsx | 0 | 0 | 0 | 0 | 1-196
page.tsx | 0 | 0 | 0 | 0 | 1-23
app/login | 0 | 0 | 0 | 0 |
LoginForm.tsx | 0 | 0 | 0 | 0 | 1-63
page.tsx | 0 | 0 | 0 | 0 | 1-28
app/productos/[id] | 0 | 0 | 0 | 0 |
QuoteForm.tsx | 0 | 0 | 0 | 0 | 1-160
page.tsx | 0 | 0 | 0 | 0 | 1-126
app/registro | 0 | 0 | 0 | 0 |
RegisterForm.tsx | 0 | 0 | 0 | 0 | 1-122
page.tsx | 0 | 0 | 0 | 0 | 1-32
components | 0 | 0 | 0 | 0 |
Footer.tsx | 0 | 0 | 0 | 0 | 1-39
LogoutButton.tsx | 0 | 0 | 0 | 0 | 1-22
```

Asignatura	Datos del alumno	Fecha
Generalización de Código de Automatización en Desarrollo de Software con IA	Apellidos: Vegas Salazar	29/06/2026
	Nombre: Juan Carlos	

```

PS C:\Users\jcvieg\Desktop\repo_local\master_universitario_2026\generaciondecodigo\actividad\latamtradox-generativecode> docker run --rm latamtradox-builder npm run test:coverage

app/dashboard/comprador/asesorias | 0 | 0 | 0 | 0 | 1-72
app/dashboard/proveedor | 0 | 0 | 0 | 0 | 1-85
app/dashboard/proveedor/documentos | 0 | 0 | 0 | 0 | 1-186
app/dashboard/proveedor/ordenes | 0 | 0 | 0 | 0 | 1-181
app/dashboard/proveedor/productos | 0 | 0 | 0 | 0 | 1-196
app/login | 0 | 0 | 0 | 0 | 1-23
app/productos/[id] | 0 | 0 | 0 | 0 | 1-160
app/registro | 0 | 0 | 0 | 0 | 1-126
app/registro | 0 | 0 | 0 | 0 | 1-122
components | 0 | 0 | 0 | 0 | 1-39
footer | 0 | 0 | 0 | 0 | 1-22
loginButton | 0 | 0 | 0 | 0 | 1-63
navbar | 0 | 0 | 0 | 0 | 1-52
productCard | 88.16 | 81.48 | 84.61 | 88.16 | 22,25,61,62,79,82
auth.ts | 87.8 | 88.23 | 85.71 | 87.8 | 100 | 100 | 100 | 100 | 100
prisma.ts | 100 | 0 | 100 | 100 | 100 | 100 | 100 | 100 | 100
purchaseOrder.ts | 100 | 87.5 | 100 | 100 | 100 | 100 | 100 | 100 | 100
stripe.ts | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0
validation.ts | 100 | 100 | 100 | 100 | 100 | 100 | 100 | 100 | 100

Test Suites: 4 passed, 4 total
Tests: 40 passed, 40 total
Snapshots: 0 total
Time: 2.339 s
Ran all test suites.

PS C:\Users\jcvieg\Desktop\repo_local\master_universitario_2026\generaciondecodigo\actividad\latamtradox-generativecode>

```

Figuras 1, 2 y 3: Ejecución de las pruebas automatizadas dentro del contenedor Docker mediante el comando `npm run test:coverage`. Se observa que las 40 pruebas de las 4 suites pasaron con éxito (100%), alcanzando una cobertura de código excelente en los archivos críticos de lógica de negocio (`validation.ts`, `purchaseOrder.ts` y `prisma.ts` al 100%, y `auth.ts` al 87.8%)

Asignatura	Datos del alumno	Fecha
Generalización de Código de Automatización en Desarrollo de Software con IA	Apellidos: Vegas Salazar	29/06/2026
	Nombre: Juan Carlos	

Parte A — Tests E2E con Playwright ✅ (6/6 verdes)

Archivo	Contenido
<code>playwright.config.ts</code>	<code>baseURL</code> por <code>E2E_BASE_URL</code> (default localhost:3000), <code>trace: on</code> , <code>screenshots</code> en fallo, <code>slowMo</code> por env, Chromium del sistema, y <code>webServer</code> que siembra la BD y arranca la app.
<code>e2e/runner.ts</code>	Adaptador que ejecuta la definición compartida de pasos con la API de Playwright.
<code>e2e/login.spects</code>	Login de los 3 roles + control de acceso (buyer no entra a admin, anónimo redirige).
<code>e2e/happy-path.spects</code>	Flujo completo extremo a extremo consumiendo <code>buildDemoFlow()</code> .

Verificado: `docker run --rm latamtrader-builder npm run e2e` → **6 passed** (incluye el happy path: proveedor publica → admin aprueba → comprador cotiza con pago → admin acepta y genera OC → proveedor avanza la orden → comprador ve seguimiento → asesoría pagada). Datos idempotentes (producto `Café E2E <timestamp>`), trace activado, `E2E_BASE_URL` para apuntar a la URL desplegada.

Parte B — Overlay de demo ✅

Archivo	Contenido
<code>src/lib/demoFlow.ts</code>	Definición única del flujo (pasos: acción + testid + valor + descripción) que consumen Playwright y el overlay.
<code>src/components/DemoRunner/</code>	Overlay con ▶/⏸/■, barra de progreso y log; resalta el elemento activo; persiste en <code>localStorage</code> y continúa tras cada navegación; se activa con <code>?demo=1</code> (desactiva con <code>?demo=0</code>), oculto por defecto.

Activación: `http://localhost:3000/?demo=1` → Play.

Otros cambios

- `data-testid` estables en login, catálogo, cotización, moderación admin, cotizaciones admin, órdenes del proveedor y asesorías (sin alterar diseño).
- `Dockerfile`: Chromium + libs en la etapa `builder`, `PLAYWRIGHT_SKIP_BROWSER_DOWNLOAD` y `PLAYWRIGHT_CHROMIUM_PATH`.
- `package.json`: scripts `e2e`, `e2e:headed`, `e2e:ui`, `e2e:server` + `@playwright/test`.
- `README.md`: secciones nuevas de E2E y overlay.
- Jest intacto: 43/43 verdes (añadí `__tests__/demoFlow.test.ts`).

Figura 4: Resumen de la ejecución exitosa (6/6) de las pruebas E2E con Playwright documentado por la IA.

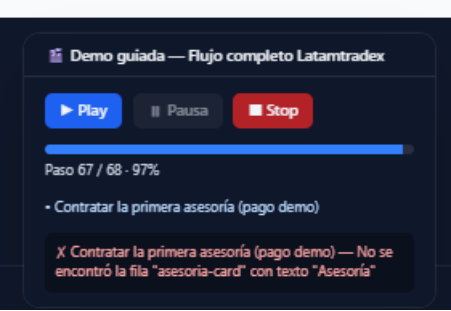


Figura 5: Interfaz del Overlay Demo en ejecución fallando en el paso 68 por un selector del DOM no encontrado, lo cual sirvió como base para la depuración guiada con la IA.

Asignatura	Datos del alumno	Fecha
Generalización de Código de Automatización en Desarrollo de Software con IA	Apellidos: Vegas Salazar	29/06/2026
	Nombre: Juan Carlos	

Conversaciones relevantes con la IA

Configuración del entorno dentro de Docker:

Le indiqué a la IA (Claude Code) que instalara Jest y React Testing Library asegurando que todo ocurriera dentro de Docker. La IA decidió inteligentemente usar next/jest para la configuración y reconstruir la etapa builder del Dockerfile en lugar de instalar dependencias efímeras en el contenedor de producción, lo que demostró una gran comprensión de la arquitectura.

Refactorización proactiva de la IA

Al solicitar las pruebas para la máquina de estados, la IA decidió extraer la lógica a un archivo independiente (lib/purchaseOrder.ts) para que las pruebas evaluaran el código real y no una copia duplicada. Esto mejoró enormemente la mantenibilidad del código

Depuración guiada

- Fallo de compilación (Type Error): Tras la refactorización, el next build falló por un error de tipado estricto en TypeScript (statusLabels[order.status]). Le pasé el log del error a la IA y ella ajustó el indexado con un cast.
- Fallo de prueba (Bug en el test UI): Una prueba falló (getAllByText('✓')) porque esperaba encontrar elementos completados en el estado inicial GENERATED (donde hay 0). La IA depuró su propio test cambiando getAllByText (que arroja error con 0 coincidencias) por queryAllByText. Esto permitió llegar a las 40/40 pruebas superadas.

Asignatura	Datos del alumno	Fecha
Generalización de Código de Automatización en Desarrollo de Software con IA	Apellidos: Vegas Salazar	29/06/2026
	Nombre: Juan Carlos	

Generación de Pruebas E2E y Depuración de Interfaz (Playwright):

Se solicitó a la IA generar un entorno de pruebas E2E utilizando Playwright que corriera nativamente dentro de la imagen de Docker. Como valor añadido, la IA generó un "Overlay" interactivo. Durante la ejecución de la prueba en vivo, se detectó un fallo en el paso 68/68 ("*No se encontró la fila asesoria-card*"). Le proporcioné a la IA el log visual del error y esta dedujo que faltaba inyectar el atributo data-testid en el componente de Asesorías. Esto demuestra cómo la IA no solo genera código, sino que es capaz de depurar selectores del DOM basándose en la retroalimentación de los tests que fallan.

Análisis de resultados y conclusiones

La automatización de pruebas mediante IA ha demostrado ser altamente efectiva. La ejecución arrojó un resultado de 40 pruebas superadas de forma veloz (~2.3 segundos), logrando una cobertura de sentencias y funciones del 100% en la lógica central (purchaseOrder.ts, validation.ts). Al medir esta cobertura, garantizamos que las áreas críticas del software fueron evaluadas.

A nivel técnico, destaco los siguientes puntos:

- Ventaja de la IA en la generación de casos: La IA fue capaz de concebir escenarios que podrían pasar desapercibidos en un diseño de pruebas manual (ej. intentar transiciones de estados hacia atrás en una Orden de Compra o

Asignatura	Datos del alumno	Fecha
Generalización de Código de Automatización en Desarrollo de Software con IA	Apellidos: Vegas Salazar	29/06/2026
	Nombre: Juan Carlos	

auto-registrarse como ADMIN), ampliando drásticamente la cobertura de las pruebas.

- Limitaciones y supervisión necesaria: La IA introdujo un "bug" en su propio código de prueba (el uso de `getAllByText` frente a `queryAllByText` al buscar cero elementos). Esto evidencia que el código generado por IA no es infalible y requiere que el desarrollador interprete correctamente los logs de la consola para guiar a la IA hacia la corrección.
- Eficiencia en DevOps: Ejecutar las pruebas directamente en un contenedor efímero (`docker run --rm latamtradex-builder`) garantiza que el entorno de validación sea idéntico al de producción. Esto sienta una base sólida para integrar estas pruebas en un futuro pipeline de Integración y Despliegue Continuo (CI/CD), asegurando la calidad del código ante futuros cambios.
- Generación de Pruebas E2E y Depuración de Interfaz (Playwright): Se solicitó a la IA generar un entorno de pruebas E2E utilizando Playwright que corriera nativamente dentro de la imagen de Docker. Como valor añadido, la IA generó un "Overlay" interactivo. Durante la ejecución de la prueba en vivo, se detectó un fallo en el paso 68/68 ("No se encontró la fila asesoria-card"). Le proporcioné a la IA el log visual del error y esta dedujo que faltaba inyectar el atributo `data-testid` en el componente de Asesorías. Esto demuestra cómo la IA no solo genera código, sino que es capaz de depurar selectores del DOM basándose en la retroalimentación de los tests que fallan.